Process & Decision Documentation

This document is used to make your design and development process visible. At this stage of your academic career, you are expected not only to produce finished work, but to articulate how decisions were made, how ideas changed, and how collaboration (for the assignments that include group work) unfolds

In professional and co-op contexts, employers do not only evaluate your final projects in your portfolio. They often ask candidates to explain their process, justify trade-offs, reflect on iteration, and describe their roles within a team.

You will need to submit a modification of this document for every group assignment (A1 – A3) and a shorter version for your individual assignments (Side Quests and A4).

Project/Assignment Decisions

Side Quests and A4 (Individual Work)

I decided to use ChatGPT to help generate the code because I am not the strongest in coding and needed to complete this side quest relatively fast as I have multiple things going on in my life right now. Using ChatGPT helped me complete the side quest on a timely manner and made my work better than what it would have been if I hadn't used it. The code GenAI produced was of good quality and would have not looked like it does if I had done it myself.

Role-Based Process Evidence

```
let debugMode = false;
let moonGravity = false;
let showHitboxes = false;

const NORMAL_GRAVITY = 10;
const MOON_GRAVITY = 2;
```

```
function keyPressed() {
  startMusicIfNeeded();

  // Toggle debug screen
  if (key === "`") {
    debugMode = !debugMode;
  }

  if (debugMode) {
    // Toggle moon gravity
    if (key === "m" || key === "M") {
      moonGravity = !moonGravity;
      world.gravity.y = moonGravity ? MOON_GRAVITY : NORMAL_GRAVITY;
    }

    // Toggle hitbox visibility
    if (key === "h" || key === "H") {
      showHitboxes = !showHitboxes;

      player.debug = showHitboxes;
      sensor.visible = showHitboxes;
    }
  }
}
```

\

Code provided by ChatGPT that was added for the debug toggle interaction instructions

```
// --- KEEP IN VIEW ---
player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2);

if (player.pos.y < FRAME_H / 2) {
  player.pos.y = FRAME_H / 2;
  player.vel.y = 0; // stop upward drift
}      You, 39 minutes ago • Uncommitted changes
```

Code added to ensure fox cannot exceed the screen height and remains in the frame when jumping

```
// --- DEBUG OVERLAY ---
if (debugMode) {
  camera.off();

  fill(0, 150);
  noStroke();
  rect(10, 10, 180, 90);

  fill(255);
  textSize(10);
  text("DEBUG MODE", 20, 25);

  text("` : Toggle Debug", 20, 40);
  text("M : Moon Gravity", 20, 55);
  text("H : Hitboxes", 20, 70);

  text("Gravity: " + (moonGravity ? "MOON" : "NORMAL"), 20, 85);

  camera.on();
}
```

Code added for the debug screen to appear upon interaction with the keys

```
// --- DEBUG INSTRUCTION TEXT ---
if (!debugMode) {
  camera.off();

  push();
  resetMatrix();
  textSize(8);
  fill(0);

  textAlign(LEFT, TOP);
  text("Press '~' for debug screen", 10, 10);
  pop();

  camera.on();
}
```

Code generated by ChatGPT for the instruction on the screen to initiate the debug toggle screen but altered by me to make the text more visible and accessible

Entry Header

Name: Mercedes Wilson

Role(s): Coder and ChatGPT prompter

Primary responsibility for this work: Prompt ChatGPT to give me an accurate debug screen

Goal of Work Session

Briefly describe what you were trying to accomplish during this phase of the assignment:

- Add code for a debug screen
- Ensure the code ran and made the most sense for the game
- Add text instruction for the debug screen

Tools, Resources, or Inputs Used

- ChatGPT
- Tutorial notes
- Side quest instructions

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: March 20th, 2026

Tool Disclosure: ChatGPT 5.2

Purpose of Use: Coding help

Summary of Interaction: ChatGPT helped code the debug screen and helped trouble shoot when the code wasn't working.

Human Decision Point(s): I altered some of the code that GenAI provided for the text as it was unnecessary and looked sloppy.

Integrity & Verification Note: I ensured I was following the side quest instructions and the tutorial slide on LEARN.

Scope of GenAI Use: GenAI did not write any aspects of the writing portions for me.

Limitations or Misfires: ChatGPT could not get the text instruction right as it kept on giving me incorrect code instructions and then produced instructions that was hard to read.

Summary of Process (Human + Tool)

Describe what you did, focusing on process rather than outcome:

- Iteration between ChatGPT's responses
- Altering the visuals of the text that GenAI produced
- Wrote the README section

Decision Points & Trade-offs

Describe one or two key decisions you made:

- I altered the code using ChatGPT to ensure the fox did not go off the screen when it jumped in the moon gravity mode

Verification & Judgement

Explain how you evaluated whether your decision or change was appropriate:

- Playtesting and iterations between ChatGPT's responses
- Re-reading side quest instructions

Limitations, Dead Ends, or Open Questions

- Adding the text instructions to the game screen took multiple tries as ChatGPT did not provide the right code in the responses. It took multiple tries to get the code to work and then I had to alter it myself afterwards to make it more visually appealing.

Appendix

add a debug screen that allows you to toggle moon gravity on/off, and at least one other potential feature you might need in developing a game, using this code:

I can definitely help with that—but I need to see your code first.

Please paste the code you're working with (especially your game loop, physics/gravity logic, and input handling), and I'll:

- Add a **debug screen / overlay**
- Implement a **toggle for moon gravity (on/off)**
- Add at least one more useful dev feature (e.g., hitbox visualization, FPS counter,

/* Week 9 — Example 3: Adding Sound & Music Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han Date: Mar. 19, 2026 Controls: A or D (Left / Right Arrow) Horizontal movement W (Up Arrow) Jump Space Bar Attack Tile key: g = groundTile.png (surface ground) d = groundTileDeep.png (deep ground, below surface) = empty (no sprite) */ let player; let playerImg, bgImg; let jumpSfx, musicSfx; let musicStarted = false; let playerAnis = { idle: { row: 0, frames: 4, frameDelay: 10 }, run: { row: 1, frames: 4, frameDelay: 3 }, jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 }, attack: { row: 3, frames: 6, frameDelay: 2 }, }; let ground, groundDeep; let groundImg, groundDeepImg; let attacking = false; // track if the player is attacking let attackFrameCounter = 0; // tracking attack animation // --- TILE MAP --- // an array that uses the tile key to create the level let level = [" ", " ", " ", " ", " ", " ggg ", "gggggggggggggg", // surface ground "ddddddddddddddd", // deep ground]; // --- LEVEL CONSTANTS --- // camera view size const VIEWW = 320, VIEWH = 180; // tile width & height const TILE_W = 24, TILE_H = 24; // size of individual animation frames const FRAME_W = 32, FRAME_H = 32; // Y-coordinate of player start (4 tiles above the bottom) const MAP_START_Y = VIEWH - TILE_H * 4; // gravity const GRAVITY = 10; function preload() { // --- IMAGES --- playerImg = loadImage("assets/foxSpriteSheet.png"); bgImg = loadImage("assets/combinedBackground.png"); groundImg = loadImage("assets/groundTile.png"); groundDeepImg = loadImage("assets/groundTileDeep.png"); // --- SOUND --- if (typeof loadSound === "function") { jumpSfx = loadSound("assets/sfx/jump.wav"); musicSfx = loadSound("assets/sfx/music.wav"); } } function setup() { // pixelated rendering with autoscaling new Canvas(VIEWW, VIEWH, "pixelated"); // needed to correct an visual artifacts from attempted antialiasing allSprites.pixelPerfect = true; world.gravity.y = GRAVITY; // Try to start background music immediately. if (musicSfx) musicSfx.setLoop(true); startMusicIfNeeded(); // --- TILE GROUPS --- ground = new Group(); ground.physics = "static"; ground.img = groundImg; ground.tile = "g"; groundDeep

```
= new Group(); groundDeep.physics = "static"; groundDeep.img = groundDeepImg; groundDeep.tile = "d"; // a Tiles object creates a level based on the level map array (defined at the beginning) new Tiles(level, 0, 0, TILE_W, TILE_H); // --- PLAYER --- player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H); // create the player player.spriteSheet = playerImg; // use the sprite sheet player.rotationLock = true; // turn off rotations (player shouldn't rotate) // player animation parameters player.anis.w = FRAME_W; player.anis.h = FRAME_H; player.anis.offset.y = -4; // offset the collision box up player.addAnis(playerAnis); // add the player animations defined earlier player.ani = "idle"; // default to the idle animation player.w = 18; // set the width of the collsion box player.h = 20; // set the height of the collsion box player.friction = 0; // set the friciton to 0 so we don't stick to walls player.bounciness = 0; // set the bounciness to 0 so the player doesn't bounce // --- GROUND SENSOR --- for use when detecting if the player is standing on the ground sensor = new Sprite(); sensor.x = player.x; sensor.y = player.y + player.h / 2; sensor.w = player.w; sensor.h = 2; sensor.mass = 0.01; sensor.removeColliders(); sensor.visible = false; let sensorJoint = new GlueJoint(player, sensor); sensorJoint.visible = false; } function startMusicIfNeeded() { if (musicStarted || !musicSfx) return; const startLoop = () => { if (!musicSfx.isPlaying()) musicSfx.play(); musicStarted = musicSfx.isPlaying(); }; // Some browsers require a user gesture before audio can start. const maybePromise = userStartAudio(); if (maybePromise && typeof maybePromise.then === "function") { maybePromise.then(startLoop).catch(() => {}); } else { startLoop(); } } function keyPressed() { startMusicIfNeeded(); } function mousePressed() { startMusicIfNeeded(); } function touchStarted() { startMusicIfNeeded(); return false; } function draw() { // --- BACKGROUND --- camera.off(); imageMode(CORNER); image(bgImg, 0, 0, bgImg.width, bgImg.height); camera.on(); // --- PLAYER CONTROLS --- // first check to see if the player is on the ground let grounded = sensor.overlapping(ground); // -- ATTACK INPUT -- if (grounded && !attacking && kb.presses("space")) { attacking = true; attackFrameCounter = 0; player.vel.x = 0; player.ani.frame = 0; player.ani = "attack"; player.ani.play(); // plays once to end } // -- JUMP -- if (grounded && kb.presses("up")) { player.vel.y = -4; if (jumpSfx) jumpSfx.play(); } // --- STATE MACHINE --- if (attacking) { attackFrameCounter++; // Attack lasts ~6 frames * frameDelay 2 = 12 cycles (adjust if needed) if (attackFrameCounter > 12) { attacking = false; attackFrameCounter = 0; } } else if (!grounded) { player.ani = "jump"; player.ani.frame = player.vel.y < 0 ? 0 : 1; } else { player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle"; } // --- MOVEMENT --- if (!attacking) { player.vel.x = 0; if (kb.pressing("left")) { player.vel.x = -1.5; player.mirror.x = true; } else if (kb.pressing("right")) { player.vel.x = 1.5; player.mirror.x = false; } } // --- KEEP IN VIEW --- player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2); }
```

Nice, this is a solid base already. Let's add a **debug screen overlay** with:

1. ✅ Toggle **moon gravity (on/off)**
2. ✅ Toggle **hitbox/sensor visibility** (super useful for debugging collisions)

🔧 What we'll add

- A `debugMode` toggle (press **~ key**)
- A small on-screen debug panel
- Key controls:
  - `M` → toggle moon gravity 🌙
  - `H` → toggle hitboxes

✅ 1. Add these variables at the top

Put this near your other globals:

```
let debugMode = false;
let moonGravity = false;
let showHitboxes = false;

const NORMAL_GRAVITY = 10;
const MOON_GRAVITY = 2;
```

✅ 2. Update `keyPressed()` to handle debug controls

Add this inside your existing `keyPressed()` function:

```
function keyPressed() {
  startMusicIfNeeded();

  // Toggle debug screen
  if (key === "`") {
    debugMode = !debugMode;
  }

  if (debugMode) {
    // Toggle moon gravity
    if (key === "m" || key === "M") {
      moonGravity = !moonGravity;
      world.gravity.y = moonGravity ? MOON_GRAVITY : NORMAL_GRAVITY;
    }

    // Toggle hitbox visibility
    if (key === "h" || key === "H") {
      showHitboxes = !showHitboxes;
```

```
      player.debug = showHitboxes;
      sensor.visible = showHitboxes;
    }
  }
}
```

✅ 3. Add Debug UI to `draw()`

At the **very end of your `draw()` function**, add:

```
// --- DEBUG OVERLAY ---
if (debugMode) {
  camera.off();

  fill(0, 150);
  noStroke();
  rect(10, 10, 180, 90);

  fill(255);
  textSize(10);
  text("DEBUG MODE", 20, 25);

  text("` : Toggle Debug", 20, 40);
  text("M : Moon Gravity", 20, 55);
  text("H : Hitboxes", 20, 70);

  text(
    "Gravity: " + (moonGravity ? "MOON" : "NORMAL"),
    20,
    85
  );

  camera.on();
}
```

🎮 What this gives you

When you run the game:

- Press **~ (backtick)** → opens debug panel
- Press **`M`** → switches gravity (floaty moon jumps 👀)
- Press **`H`** → shows collision boxes

💡 Why this is useful (for your assignment)

This hits the requirement of:

- ✓ Debug screen
- ✓ Physics testing (gravity)
- ✓ Development tool (hitboxes)

If you want to go one level further (and impress your prof), you could also add:

- Player position display (x/y)
- FPS counter
- Teleport key

If you want, I can upgrade this into a **clean toggleable dev menu with sliders/buttons** instead of keyboard controls 👍

now add a text instruction to the main screen that says "press '~' for debug screen".

Easy win—just render a small UI hint on the main screen (with the camera off so it stays fixed).

✅ Add this inside your `draw()` function

Place it **after your background is drawn** (or anywhere outside `camera.on()` world rendering, but before debug overlay is fine):

```
// --- DEBUG INSTRUCTION TEXT ---
camera.off();

fill(255);
textSize(10);
textAlign(RIGHT);
text("Press '~' for debug screen", VIEWW - 10, VIEWH - 10);

camera.on();
```

🎯 What this does

- Keeps the text anchored to the screen (not the game world)
- Places it neatly in the **bottom-right corner**
- Always visible so users know the debug feature exists

If you want it to feel more “game-like,” I can also make it:

- Fade in/out
- Only show at the start
- Styled like pixel UI 👀

that didn't work

Yeah—that can happen depending on **where it's placed** and how your camera/UI flow is set up. Let's fix it cleanly.

✅ 🔥 Correct way to add the instruction text

Put this **at the VERY END of your `draw()` function**, *after everything else* (including your debug overlay):

```
// --- DEBUG INSTRUCTION TEXT ---
camera.off();

push();
fill(255);
stroke(0);
strokeWeight(2);
textSize(10);
textAlign(RIGHT, BOTTOM);
text("Press '~' for debug screen", VIEWW - 8, VIEWH - 6);
pop();

camera.on();
```

💡 Why your first attempt didn't show

Common issues:

- It was drawn **before something else covered it** (like background redraw)
- Camera was still **on**, so text moved off-screen
- No `stroke`, so it blended into the background

🎯 What this version fixes

- ✅ Always on top (drawn last)
- ✅ Anchored to screen (camera off)
- ✅ Readable (white text + black outline)
- ✅ Proper alignment (bottom-right corner)

If it *still* doesn't show, tell me and I'll debug it with you—could be something subtle like text font loading or canvas scaling.

It still is not appearing on the screen

Alright—if it's still not showing, the issue is almost certainly **text not rendering at all** (which can happen in p5play setups if a font isn't explicitly set or text gets overridden).

Let's force it to work 👇

✅ ✅ Reliable Fix (this WILL show)

1. Add this to your `setup()`:

```
textFont("monospace");
```

---

2. Then replace your instruction text with this at the VERY END of `draw()`:

```
// --- DEBUG INSTRUCTION TEXT ---
camera.off();

push();

// force clear styling
resetMatrix();
textFont("monospace");
textSize(12);
fill(255);
stroke(0);
strokeWeight(3);

// draw in a very obvious position first (top-left)
textAlign(LEFT, TOP);
text("Press '~' for debug screen", 10, 10);

pop();

camera.on();
```

🔍 Why this works (what was going wrong)

One (or more) of these was happening:

- ❌ No font set → text silently fails sometimes

- ❌ Camera transform interfering
- ❌ Text drawn off-screen due to scaling
- ❌ UI getting covered by later renders

This version:

- Resets transformations (`resetMatrix()`)
- Forces a font
- Draws in **top-left (guaranteed visible)**
- Uses thick stroke so it can't blend in

🧪 If you STILL don't see it

Try this quick test:

```
console.log("draw running");
```

If that prints but no text shows, then it's 100% a **rendering/font issue**, and I'll give you a fallback using `createDiv()` (HTML overlay instead).

If you want, I can also make it look like a **clean pixel-art UI hint** once it's working 👍

now make it so that the instruction disapears when the dubug screen is visible

Perfect—that's a simple logic tweak.

Right now your instruction text is always drawing. You just need to **only draw it when `debugMode` is OFF**.

✅ Update your instruction text code

Find the block you added in `draw()` and wrap it like this:

```
// --- DEBUG INSTRUCTION TEXT ---
if (!debugMode) {
  camera.off();

  push();
  resetMatrix();
  textFont("monospace");
```

```
  textSize(12);
  fill(255);
  stroke(0);
  strokeWeight(3);

  textAlign(LEFT, TOP);
  text("Press '~' for debug screen", 10, 10);
  pop();

  camera.on();
}
```

🎯 What this does

- ✅ Shows instruction during normal gameplay
- ✅ Automatically hides it when debug screen is open
- ✅ Keeps your UI clean and non-redundant

If you want to polish it further, you could:

- Fade it out after a few seconds
- Or only show it once at the start

Just tell me 👍

now how do I make it so that the fox does not go off the screen in moon gravity mode

Good question—this happens because with **low gravity**, your fox can float higher and escape your current constraints (you're only constraining **x**, not **y**).

✅ 🔗 Fix: Constrain the player's Y position

Right now you have this:

```
player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2);
```

👉 You need to also constrain **Y** so the player can't float off the top (or bottom).

✅ Add this right below your existing constraint:

```
player.pos.y = constrain(
  player.pos.y,
  FRAME_H / 2,          // top boundary
  VIEWH - FRAME_H / 2   // bottom boundary
);
```

💡 Optional (better feel for moon gravity)

If you want it to feel more natural (not like hitting an invisible ceiling), you can instead **soft clamp** the top:

```
if (player.pos.y < FRAME_H / 2) {
  player.pos.y = FRAME_H / 2;
  player.vel.y = 0; // stop upward drift
}
```

🎯 Why this works

- Moon gravity = slower falling → player can drift upward longer
- Without a Y constraint → they can leave the visible area
- This keeps them **inside the camera view at all times**

🚀 Bonus (if you want extra polish)

You could also:

- Add a **"ceiling" tile** instead of a hard clamp
- Or make the camera follow the player vertically

If you want, I can help you turn moon gravity into a **full mechanic** (like double jump or float mode 👀)